

Network+ & SOC Ingestion Pipeline Master Guide

Author: Matt (optomist-5) | **Domain:** Threat Detection & Security Engineering | **Date:** September 2026

Module 1: Wireless Security & 802.11 Protocol Mechanics

Modern wireless communications operate under the IEEE 802.11 standard. Understanding wireless vulnerabilities requires analyzing the distinction between frame types, encryption protocols, and management message integrity.

802.11 Frame Architecture & Management Vulnerabilities

802.11 traffic is divided into three functional frame types:

1. **Data Frames:** Carry payload data across the wireless medium.
2. **Control Frames:** Assist with data delivery (e.g., RTS, CTS, ACK).
3. **Management Frames:** Supervise network joins, leaves, and capability advertisements (e.g., Beacons, Probe Requests/Responses, Authentication, Association, Disassociation, and Deauthentication).

The Core Vulnerability: In legacy Wi-Fi standards (802.11a/b/g/n/ac), management frames are transmitted in plaintext and lack cryptographic signatures. An attacker can construct spoofed 802.11 management frames containing the target client's or Access Point's MAC address to force immediate disconnection.

Attack Vector Deep Dive & Enterprise Defense

Deauthentication Attack: A active Layer 2 Denial of Service (DoS) attack where forged deauthentication frames force client disconnections. Attackers leverage this to kick users off legitimate APs and force them to connect to an 'Evil Twin' rogue AP broadcasting an identical SSID.

Protected Management Frames (PMF / 802.11w): Adds cryptographic protection (via AES-CMAC or HMAC-SHA256) to management frames. Clients drop unauthenticated deauthentication frames.

WPA3 Mandate: WPA3 strictly mandates 802.11w PMF support, eliminating plaintext deauthentication attacks.

Module 2: Network Traffic Capture & Baseline Engineering

Full Packet Capture (FPC) records every payload byte traversing a link. While invaluable during forensic investigations, enterprise links (10Gbps+) generate terabytes of data daily, overwhelming SIEM storage arrays.

SOC Ingestion Trade-off: Security Operations Centers employ statistical sampling to archive baseline captures (e.g., 2,500 packets per week) to establish network noise baselines, validate Snort/Zeek detection rules, and retain audit logs without exhausting disk quotas.

Python Scapy Implementation Code

The complete operational packet capture script executed on interface en0:

```
#!/usr/bin/env python3
import os, sys, time
from datetime import datetime
from scapy.all import sniff, wrpcap

INTERFACE = 'en0'           # macOS Wi-Fi Interface
MAX_PACKETS = 2500          # Target Sample Size
TIMEOUT = 300               # 5-minute timeout guard
OUTPUT_DIR = './captures'

def check_privileges():
    if os.geteuid() != 0:
        print('[ - ] Root/sudo required for raw sockets.', file=sys.stderr)
```

```

        sys.exit(1)

def run_capture():
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    timestamp = datetime.now().strftime('%Y-%m-%d_%H-%M-%S')
    out_file = os.path.join(OUTPUT_DIR, f'sample_{timestamp}.pcap')
    packets = sniff(iface=INTERFACE, count=MAX_PACKETS, timeout=TIMEOUT)
    wrpcap(out_file, packets)
    print(f'[+] Saved {len(packets)} packets to {out_file}')

if __name__ == '__main__':
    check_privileges()
    run_capture()

```

Module 3: Unix Administration & Daemon Scheduling

Automating security scripts requires proper Unix privilege management and background execution daemons.

Raw Socket Privileges: Capturing packets bypasses standard OS socket abstractions, opening raw network drivers. This requires EUID 0 (root). Running scripts without sudo results in permission denied errors.

Root Crontab Execution: Standard user cron jobs fail when attempting raw packet capture. The job must be registered under the root user crontab (sudo crontab -e).

Cron Expression Breakdown (0 2 * * 0):

- 0: Minute 0
- 2: 02:00 AM
- *: Every day of month
- *: Every month
- 0: Sunday

Command Entry:

```
0 2 * * 0 /Users/mq/pcap-auto-sampler/venv/bin/python3 /Users/mq/pcap-auto-sampler/capture.py >>
/var/log/pcap_sampler.log 2>&1
```

Module 4: Version Control, Authentication & Identity

Modern DevSecOps workflows mandate strict access controls, repository hygiene, and license tracking.

Personal Access Tokens (PAT): GitHub deprecated basic password authentication over HTTPS for command-line operations. Developers must generate cryptographically scoped PATs (e.g., ghp_...) with specific permissions (repo).

Repository Hygiene (.gitignore): Raw .pcap files and virtual environments (venv/) must be excluded from Git tracking to avoid committing sensitive user traffic payloads or bloated binaries.

MIT Licensing: A permissive open-source license protecting authors from liability while permitting public review and professional evaluation.

Module 5: Comprehensive Troubleshooting & Root Cause Matrix

A detailed retrospective log documenting operational errors encountered during execution, technical root causes, and resolutions:

Error Observed	Technical Root Cause	Engineering Resolution
zsh: command not found: # zsh: 2.5.0 not found	Multi-line configuration text pasted directly into Zsh prompt; shell evaluated text as commands.	Implemented cat << 'EOF' > file heredoc streaming to write clean disk files.
Prompt stuck at heredoc>	Unclosed quotes or pasting tutorial text into active heredoc stream.	Issued Ctrl+C (SIGINT) to interrupt Zsh execution and reset terminal prompt.
Terminal un-responsive on password entry	macOS Terminal intentionally suppresses character echoing on sudo for security.	Pasted password via Cmd+V and pressed Enter regardless of blank display.
remote: Invalid username or token	Basic account password used for Git HTTPS push instead of Personal Access Token.	Generated classic PAT with repo scope on GitHub; pasted token string as password.
fatal: Need to specify how to reconcile divergent branches	Remote repo initialized with web-generated LICENSE, creating disconnected commit trees.	Set git config pull.rebase false and pulled with --allow-unrelated-histories.
sudo: 3 incorrect password attempts	Pasting GitHub PAT (ghp_...) into sudo prompt instead of local Mac password.	Entered local macOS user unlock password required by system PAM authentication.

Module 6: CompTIA Network+ & Security+ Exam Mapping

Exam Objectives Directly Reinforced by this Project:

- **Network+ Domain 2.0 (Network Operations):** Monitoring baseline traffic, capturing PCAP files, analyzing packet metrics.
- **Network+ Domain 4.0 (Network Security):** 802.11 wireless security standards (WPA2 vs WPA3, 802.11w PMF, rogue APs, deauthentication attacks).
- **Security+ Domain 1.0 (Threats & Vulnerabilities):** Wireless attack vectors, Layer 2 spoofing, DoS mechanics.
- **Security+ Domain 3.0 (Security Architecture):** Secure authentication mechanisms, API token scopes, least privilege authorization.